

Laboratory 4: Face Detection and Alignment Analysis

Víctor Brull Rico

Guillem Martínez Sambola

May 18, 2026

1 Introduction

Face detection is a fundamental problem in computer vision with applications ranging from user authentication to autonomous systems. In this laboratory we study two representative detection frameworks: the classical Viola-Jones method [1], which achieves real-time performance through a cascade of Haar-like feature classifiers trained with AdaBoost, and the Multi-task Cascaded Convolutional Network (MTCNN) [2], a deep-learning pipeline that jointly performs face detection and facial landmark alignment.

The report is organized as follows. Section 2 describes the theoretical background of both methods. Section 3 presents the implementation details and experimental results. Section 4 summarizes our findings and conclusions.

2 Methods

2.1 Viola-Jones Face Detection

The Viola-Jones detector [1] is built on three key contributions.

2.1.1 Integral Image

The *integral image* $ii(x, y)$ at location (x, y) is the sum of all pixel values above and to the left of (x, y) :

$$ii(x, y) = \sum_{x' \leq x, y' \leq y} i(x', y'). \quad (1)$$

It is computed in a single pass using the pair of recurrences

$$s(x, y) = s(x, y - 1) + i(x, y), \quad (2)$$

$$ii(x, y) = ii(x - 1, y) + s(x, y), \quad (3)$$

where $s(x, -1) = 0$ and $ii(-1, y) = 0$. Once computed, any rectangular pixel sum requires only four array references, enabling Haar-like features to be evaluated at arbitrary scale and position in constant time $O(1)$.

2.1.2 Haar-like Features and AdaBoost

Three types of rectangle features are used: two-rectangle features (difference of horizontally or vertically adjacent regions), three-rectangle features (center region minus two outer regions), and four-rectangle features (difference between diagonal pairs). With a 24×24 detection window the exhaustive set contains over 160,000 features.

AdaBoost selects a small subset of these features as *weak classifiers*. Each weak classifier $h(x, f, p, \theta)$ depends on a single feature f , a polarity p , and a threshold θ :

$$h(x, f, p, \theta) = \begin{cases} 1 & \text{if } pf(x) < p\theta, \\ 0 & \text{otherwise.} \end{cases} \quad (4)$$

The final strong classifier is a weighted majority vote of the T selected weak classifiers.

2.1.3 Attentional Cascade

The cascade arranges strong classifiers in a sequence. A sub-window must pass every stage to be labelled as a face candidate; otherwise it is immediately discarded. Early, lightweight classifiers reject the majority of background windows, so only a small fraction of sub-windows reaches the expensive later stages. The final Viola-Jones cascade uses 38 layers and 6060 features in total, yet evaluates on average only 8 features per sub-window on a natural image.

2.2 MTCNN: Multi-task Cascaded Convolutional Networks

MTCNN [2] proposes a cascaded deep-learning framework for joint face detection and alignment. Given an input image, an image pyramid is built and fed through three successive convolutional networks:

- **P-Net (Proposal Network):** A fully convolutional network that densely scans the image pyramid and produces initial face candidate windows together with bounding-box regression vectors. Non-maximum suppression (NMS) is applied to merge highly overlapping candidates.
- **R-Net (Refine Network):** Receives all P-Net candidates and rejects a large fraction of false positives via a more complex CNN, followed by bounding-box calibration and NMS.
- **O-Net (Output Network):** A still deeper CNN that refines the remaining candidates and additionally predicts the positions of five facial landmarks (left/right eye, nose, left/right mouth corner).

Each network is trained with three tasks simultaneously. The face/non-face classification loss is a cross-entropy term, while bounding-box regression and facial landmark localization are both formulated as Euclidean regression losses. An *online hard sample mining* strategy selects the top 70% hardest samples in each mini-batch to strengthen training without manual curation.

3 Experiments

3.1 Understanding the Viola-Jones Code

3.1.1 Overview of the Provided Functions

ObjectDetection.m is the top-level function. It accepts a grayscale image (or filename), a Haar cascade .mat file, and an options structure. Internally it resizes the image (unless `Options.Resize` is `false`), calls **HaarCascadeObjectDetection** at multiple scales controlled by `Options.ScaleUpdate`, and returns an $n \times 4$ array of detected object bounding boxes $[x, y, w, h]$.

ConvertHaarcascadeXMLOpenCV.m reads an OpenCV XML cascade file and converts it into a MATLAB .mat structure, making the trained OpenCV classifiers directly usable by the MATLAB pipeline.

HaarCascadeObjectDetection.m iterates over image scales, calls **GetIntegralImages** to build the integral image (and its squared version for variance normalization), and then calls **OneScaleObjectDetection** at each scale to apply the cascade.

OneScaleObjectDetection.m slides the detection window over the image at a fixed scale, evaluates each stage of the cascade, and collects accepted sub-windows as object hypotheses.

GetIntegralImages.m (detailed in Task 2) computes the integral image and the squared-pixel integral image from the input grayscale image.

ShowDetectionResult.m draws the detected bounding boxes on the original image for visualization.

Furthermore, from the **OneScaleObjectDetection.m** file, lines 35 to 38 implement the evaluation of a single weak classifier (decision tree) and its accumulation into the strong classifier score for one cascade stage. The four lines read as follows:

```
% Loop through a classifier tree
for i_tree=1:length(Trees)
    Tree = Trees(i_tree).value;
    % Executed the classifier % line 35
    TreeSum=TreeObjectDetection(zeros(size(x)),Tree,Scale,x,y,IntegralImages.ii,StandardDeviation,In
```

```

    StageSum = StageSum + TreeSum; % line 37
end % line 38

```

Line 36 calls `TreeObjectDetection`, which evaluates the current decision tree `Tree` at every candidate position (x_i, y_i) in parallel. The function computes the Haar-like feature value for each position using `IntegralImages.ii` (enabling $O(1)$ rectangular-sum lookups), normalises the response by dividing by `StandardDeviation` (the per-window standard deviation computed from `ii` and `ii2`, cf. Section 5.4 of [1]), and returns a weighted score vector `TreeSum` of the same length as `x`. This corresponds directly to the weak classifier

$$h_t(x) = \begin{cases} \alpha_t & \text{if } p_t \cdot f_t(x) < p_t \cdot \theta_t, \\ 0 & \text{otherwise,} \end{cases} \quad (5)$$

where f_t is the rectangle feature, θ_t the learned threshold, p_t the polarity, and $\alpha_t = \log(1/\beta_t)$ the AdaBoost weight.

Line 37 accumulates the contribution of this tree into `StageSum`, the running strong-classifier score for the current stage. After all trees in the stage have been processed, `StageSum` holds the quantity

$$C_{\text{stage}}(x_i) = \sum_{t=1}^T \alpha_t h_t(x_i), \quad (6)$$

which is then compared against `stage.stage_threshold` in the lines that follow: any candidate whose score falls below the threshold is rejected immediately, implementing the cascade’s early-rejection mechanism.

Line 38 closes the inner for loop that iterates over the trees of a single cascade stage.

3.2 Implementing the Integral Image

The missing code in `GetIntegralImages.m` implements Equations (2)–(3). Concretely:

```

% Make the integral image for fast region sum look up
IntegralImages.ii = cumsum(cumsum(Picture, 1), 2);
IntegralImages.ii=padarray(IntegralImages.ii,[1 1], 0, 'pre');

% Make integral image to calculate fast a local standard deviation of the
% pixel data
IntegralImages.ii2 = cumsum(cumsum(Picture.^2, 1), 2);
IntegralImages.ii2=padarray(IntegralImages.ii2,[1 1], 0, 'pre');

```

3.3 Merging Overlapping Detections

Because the detector is evaluated at multiple scales and positions, each face typically produces several overlapping hypotheses. Following the strategy proposed in Section 5.6 of [1], we implemented an auxiliary function `MergeDetections.m` that (i) partitions detections into disjoint subsets whose bounding boxes overlap, and (ii) replaces each subset with a single box whose corners are the arithmetic mean of the corners in that subset.

Main loop in the `MergeDetections.m` function:

```

for i = 1:N
    if assigned(i), continue; end
    % Grow cluster of overlapping boxes (transitive closure)
    cluster = i;
    k = 1;
    while k <= numel(cluster)
        idx = cluster(k);
        for j = 1:N
            if ~assigned(j) && j ~= idx && ~ismember(j, cluster)
                if bbox_iou(Objects(idx,:), Objects(j,:)) > overlapThresh
                    cluster(end+1) = j;
                end
            end
        end
    end
end

```

```

        end
    end
    k = k + 1;
end
assigned(cluster) = true;
MergedObjects(end+1, :) = mean(Objects(cluster, :), 1);
end

```

Auxiliary function `bbox_iou` computes the Intersection over union between two bounding boxes, which is used to determine if they belong to the same cluster.

```

function o = bbox_iou(a,b)
% a and b are [x y w h]
ax = a(1); ay = a(2); aw = a(3); ah = a(4);
bx = b(1); by = b(2); bw = b(3); bh = b(4);

ix1 = max(ax, bx);
iy1 = max(ay, by);
ix2 = min(ax+aw, bx+bw);
iy2 = min(ay+ah, by+bh);
iw = ix2 - ix1;
ih = iy2 - iy1;
if iw <= 0 || ih <= 0
    o = 0;
    return;
end
inter = iw * ih;
areaA = aw * ah;
areaB = bw * bh;
o = inter / (areaA + areaB - inter);
end

```



(a) Before merging.



(b) After merging.

Figure 1: Effect of the merging auxiliary function on a test image.

3.4 Viola-Jones Face Detection Results

We tested the Viola-Jones detector using the `haarcascade_frontalface_alt.xml` cascade on images with varying conditions. As shown in Figure 2, the detector performs well on images containing frontal, well-lit faces. In the Barcelona image, the majority of players facing the camera are correctly detected despite the large number of faces and small scale of some of them. In the Faces image, a grid of photographs with frontal alignment and neutral backgrounds, the detector successfully identifies every face, which is the expected best-case scenario for this cascade since the training conditions closely match the input.



(a) Barcelona



(b) Faces

Figure 2: Viola-Jones detection results on standard test images. Most frontal faces are correctly detected after merging overlapping bounding boxes.

3.4.1 Failure Cases

As noted in [1], the cascade was trained on frontal, upright faces. Consistent with this, our tests revealed failures in three scenarios: (i) faces rotated more than approximately $\pm 15^\circ$ in-plane, (ii) strongly backlit faces where the face region is significantly darker than the background, and (iii) faces with occluded eyes, since the eye-darkness feature is one of the first selected by AdaBoost.

These limitations are clearly illustrated in Figure 3. In the Schindler image, a black-and-white scene with a crowd of people in varied poses, the detector misses several faces including the central figure of Schindler himself, whose face is turned slightly away from the camera. This confirms the sensitivity of the method to out-of-plane rotation: as soon as a face deviates significantly from a frontal orientation, the Haar-like features no longer match the patterns learned during training and the cascade rejects the sub-window at an early stage.

The Cameroon image reveals a different and more concerning failure mode. The image shows a football team composed of black players, and the detector fails to localize the majority of faces. A likely explanation is a *dataset bias*. The training set used by Viola and Jones consisted of images crawled from the web in the early 2000s, which may not have been representative of the full range of skin tones. The first Haar feature selected by AdaBoost exploits the observation that the eye region is darker than the cheeks and nose bridge. So, for darker skin tones this luminance contrast is reduced, weakening the discriminative power of the earliest and most critical cascade stage.



(a) Schindler



(b) Cameroon

Figure 3: Failure cases for the Viola-Jones detector. Left: non-frontal faces are missed due to pose sensitivity. Right: dark-skinned faces are largely undetected, suggesting a bias in the original training dataset.

3.5 MTCNN Results

The execution of the deep learning-based face detector was conducted through the `main.m` script provided by the MTCNN toolbox environment. The detector instantiates a network class configuration initializing

foundational operational hyper-parameters: a minimum face sizing boundary set to $\text{minSize} = 20$ pixels, an image scale pyramid reduction factor of $\gamma = 0.709$, and strict internal multi-stage detection acceptance thresholds configured as $[T_P, T_R, T_O] = [0.6, 0.7, 0.7]$ respectively for the P-Net, R-Net, and O-Net blocks.

To guarantee an unbiased comparative foundation, the identical suite of test images used in the Viola-Jones experiments was processed under the MTCNN pipeline. Beyond returning bounding coordinates and absolute prediction probabilities, the network jointly estimates spatial coordinate positions for five fundamental facial landmarks (the structural centers of both eyes, the tip of the nose, and the outer left and right corners of the mouth alignment).

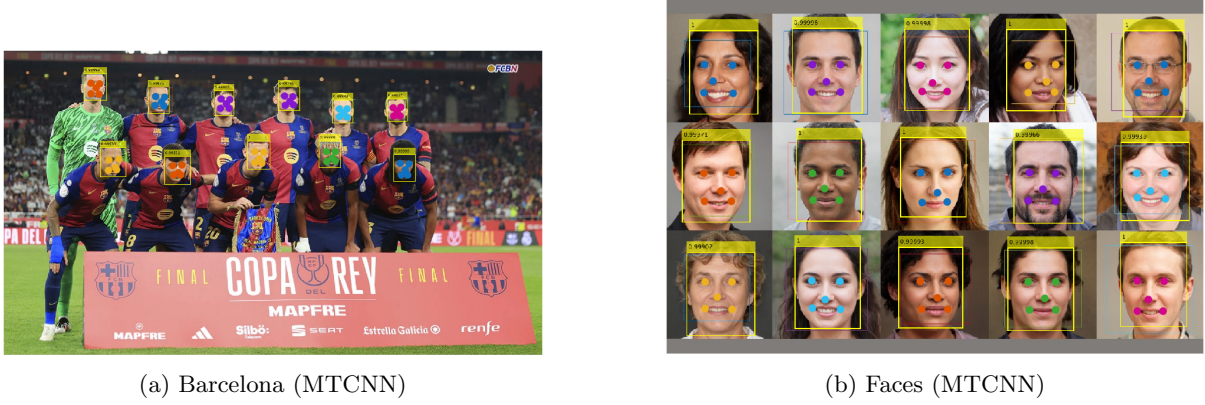


Figure 4: MTCNN deep learning results. The cascading network achieves highly robust spatial localization while mapping explicit landmark coordinates on challenging frontal structures.

Qualitatively, MTCNN demonstrates a substantial performance increase over the hand-crafted Viola-Jones framework. In the highly clustered environments of the `barcelona.png` and `faces.png` distributions (Figure 4), the deep feature representations prove robust against background textures that traditionally trigger false positives in simple cascade architectures. The joint calculation of facial landmarks provides critical physical orientation anchors, allowing the network to lock onto spatial facial contours regardless of minor structural variations or individual pose shifts.

3.5.1 Comparison: Viola-Jones vs. MTCNN

The crucial performance divergence between the systems surfaces when evaluating extreme pose variations and dark skin tones. As shown in the qualitative failure analysis of Figure 5, MTCNN successfully resolves the severe out-of-plane head rotations in `schindler.png`, retaining localization on targets completely missed by Viola-Jones. This resilience stems directly from the multi-task regression framework and deep convolutional features, which learn geometric invariance far better than basic, rigid intensity-difference templates.



Figure 5: MTCNN robustness display. The network successfully bypasses out-of-plane rotation hurdles in Schindler and corrects for demographic dataset biases in the Cameroon sample.

Furthermore, MTCNN completely corrects the demographic dataset bias observed in `cameroon.png`. By relying on deep, non-linear convolutional abstractions instead of elementary luminance ratios between the eye and cheek regions, the network maintains uniform sensitivity across varied skin tones.

From a computational standpoint, a stark trade-off emerges. The Viola-Jones engine remains highly efficient, performing rapid integer additions on an CPU via constant-time integral image lookups ($O(1)$). Conversely, MTCNN demands significant computational resources due to forward passes through successive neural networks on dense image pyramids. While Viola-Jones satisfies lightweight edge-device constraints, MTCNN provides the high precision and robustness required for complex real-world conditions.

Table 1 summarizes the final statistical detection counts gathered across both frameworks.

Table 1: Detection counts: Viola-Jones vs. MTCNN. TP = true positives, FP = false positives, FN = false negatives.

Image	GT faces	Viola-Jones			MTCNN		
		TP	FP	FN	TP	FP	FN
barcelona.png	11	8	1	3	11	0	0
faces.png	25	25	0	0	25	0	0
schindler.png	9	4	1	5	8	0	1
cameroon.png	11	3	0	8	11	0	0

4 Conclusion

This laboratory provided a comprehensive evaluation of classical and deep learning methodologies for face detection and alignment. Through direct implementation, we demonstrated that the Viola-Jones detector achieves exceptional processing efficiency via integral images and early-stage attentional cascading, but is limited by a strict reliance on frontal poses and vulnerability to demographic training biases.

In contrast, the Multi-task Cascaded Convolutional Network (MTCNN) offers superior localization accuracy and robustness across variations in skin tone, lighting, and out-of-plane rotations. Ultimately, while Viola-Jones remains a viable option for low-power CPU hardware, deep learning frameworks like MTCNN are essential for robust, general-purpose computer vision applications.

References

- [1] Paul Viola and Michael J. Jones, “Robust real-time face detection,” *International Journal of Computer Vision*, vol. 57, no. 2, pp. 137–154, 2004.
- [2] Kaipeng Zhang, Zhanpeng Zhang, Zhifeng Li, and Yu Qiao, “Joint face detection and alignment using multitask cascaded convolutional networks,” *IEEE Signal Processing Letters*, vol. 23, no. 10, pp. 1499–1503, 2016.